

RAG 기반 챗봇 앱 - PPT/문서용 자료 정리

📄 슬라이드 9: 기능별 구체적 세부 소개

🧐 1. 사용자 인터페이스 핵심 기능

A. 지능형 하이라이트 시스템

상위권 결과 자동 강조

```
// 상위 30% + 70% 이상 점수에만 하이라이트 표시
const isTopResult = index < Math.max(1, Math.ceil(retrieveResults.length * 0.3));
const isHighScore = enhancementDetails && enhancementDetails.final_score >= 0.7;
const hasEnhancement = isTopResult && isHighScore;

// 동적 하이라이트 텍스트
{enhancementDetails.final_score >= 0.8 ? '고품질' : '상위결과'}
```

하이라이트 기능들:

- 🏆 "상위결과" 배지: 상위 30% 내 고품질 검색 결과
- 🎯 점수별 색상: 80%+ 초록, 60%+ 노랑, 60%- 빨강
- 📊 진행바: 유사도 점수 시각화
- 🖱️ 호버 효과: 마우스 오버 시 상세 정보 표시

B. 스마트 출처 링크 시스템

클릭 한 번으로 정확한 위치 이동

```
const handleSourceClick = (source: SourceReference) => {
  // 지식베이스 페이지로 이동하며 정확한 위치 지정
  setKnowledgeBaseProps({
    initialDatasetId: source.datasetId, // 특정 데이터셋
    initialChunkId: source.chunkId, // 정확한 청크 위치
    initialHighlight: '', // 검색어 자동 입력 방지
  });
  setCurrentView('documents'); // 즉시 이동
};
```

출처 링크 특징:

- 🎯 정확한 위치: 청크 ID 기반 직접 이동
- 🔍 컨텍스트 보존: 원본 문서 내 정확한 위치 표시
- 📄 문서 메타데이터: 파일명, 페이지, 생성일 등 상세 정보
- 🔗 자동 스크롤: 해당 청크로 자동 이동

C. 챗 히스토리 관리 시스템

지능형 세션 복원

```
// 세션 데이터 저장 및 복원
const saveSession = (sessionId: string, data: SessionData) => {
  const sessionData = {
    messages: data.messages,
    assistantId: data.assistantId,
    sessionId: sessionId,
    timestamp: Date.now(),
    knowledgeBases: data.selectedDatasets
  };
  localStorage.setItem(`hana_session_${sessionId}`, JSON.stringify(sessionData));
};

// 브라우저 재시작 후에도 완벽 복원
useEffect(() => {
  const savedSessions = getAllSavedSessions();
  if (savedSessions.length > 0) {
    restoreSession(savedSessions[0]);
  }
}, []);
```

히스토리 기능들:

- 🗄️ 자동 저장: 모든 대화 내용 실시간 저장
- 🔄 완벽 복원: 메시지, 어시스턴트 모드, 지식베이스 상태 모두 복원
- 📱 반응형 UI: 모바일에서는 세로 배치, 데스크톱에서는 사이드바
- 🏷️ 스마트 라벨링: 대화 내용 기반 자동 제목 생성

🔧 2. 고급 상호작용 기능

A. 멀티모달 파일 업로드

드래그 앤 드롭 + 즉시 처리

```
const handleFileUpload = async (files: File[]) => {
  // 1. 임시 데이터셋 생성
  const tempDataset = await createDataset(`temp_${Date.now()}`);

  // 2. 파일 업로드 및 파싱
  await uploadDocuments(tempDataset.id, files);
  await parseDocuments(tempDataset.id);

  // 3. 즉시 대화 가능한 상태로 전환
  setSelectedDatasets([tempDataset.id]);
  setCurrentView('chat');
};
```

파일 처리 특징:

- 📄 다양한 형식: PDF, Word, Excel, 이미지, 텍스트
- ⚡ 실시간 처리: 업로드 즉시 벡터화 및 검색 가능
- 🔒 격리된 세션: 파일별 독립적인 대화 환경
- 🗑️ 자동 정리: 세션 종료 시 임시 데이터 자동 삭제

B. 실시간 품질 피드백

모든 응답에 즉시 품질 평가

```
// 응답과 동시에 품질 평가 시작
const evaluateResponse = async (question: string, answer: string) => {
  const metrics = ['accuracy', 'relevance', 'readability', 'privacy'];

  const evaluations = await Promise.all(
    metrics.map(metric =>
      evaluateWithOllama({
        question,
        answer,
        retrieved_content: sources.map(s => s.content).join('\n')
      }, metric)
    )
  );

  // 실시간 점수 표시
  displayEvaluationResults(evaluations);
};
```

품질 피드백 특징:

- 📊 4개 지표: 정확도, 관련성, 가독성, 개인정보 보호
- ⚡ 실시간 평가: 응답 생성과 동시에 품질 측정
- 📈 트렌드 분석: 시간에 따른 성능 변화 추적
- 🏆 벤치마킹: Gold Standard와 비교 평가

C. 지능형 검색 최적화

AI 중심의 검색 결과 정렬

```
// RAGFlow AI + Reranker 점수 보존
function enhanceSearchResults(chunks: any[]): any[] {
  return chunks.map((chunk, index) => {
    const originalScore = chunk.similarity || chunk.score || 0;

    return {
      ...chunk,
      similarity: originalScore, // AI 점수 그대로 보존
    };
  });
}
```

```

    _enhancement_details: {
      original_score: originalScore,
      ai_preserved: true // 인위적 조작 없음
    }
  };
}).sort((a, b) => b.similarity - a.similarity); // 높은 점수 우선
}

```

검색 최적화 특징:

- 🔄 **AI 신뢰**: RAGFlow + Reranker의 원본 점수 보존
- 🎯 **정밀 정렬**: 최종 점수 기준 정확한 순위
- 🗑️ **메타데이터 필터**: 관리정보/테이블 등 노이즈 제거
- 🔍 **컨텍스트 하이라이트**: 검색어 기준 핵심 내용 강조

🔑 3. 시스템 레벨 고급 기능

A. 동적 환경 설정

배포 후에도 설정 변경 가능

```

// 런타임 환경변수 오버라이드
const readEnv = (key: string): string | undefined => {
  const runtimeValue = (window as any).__ENV__?.[key];
  if (runtimeValue && runtimeValue !== 'undefined') {
    return runtimeValue; // 런타임 값 우선
  }
  return import.meta.env[key]; // 빌드타임 값 fallback
};

// 리랭커 모델 동적 변경
export function getRerankConfig(): { rerank_id?: string } {
  const modelName = readEnv('VITE_RAGFLOW_RERANK_MODEL');
  const isEnabled = readEnv('VITE_RAGFLOW_ENABLE_RERANK') === 'true';

  return isEnabled && modelName ? { rerank_id: modelName } : {};
}

```

B. 견고한 에러 처리

5분 타임아웃 + 다중 Fallback

```

async function ragFetch<T>(path: string, init?: RequestInit): Promise<T> {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), 300000); // 5분

  try {
    const res = await fetch(url, {

```

```

    ...init,
    signal: controller.signal,
    headers: { 'Authorization': `Bearer ${API_KEY}`, ...headers }
  });

  // JSON 파싱 실패 시 상세 에러 메시지
  if (!res.ok) {
    const errorText = await res.text();
    throw new Error(`API Error (${res.status}): ${errorText.slice(0, 200)}`);
  }

  return await res.json();
} catch (error: any) {
  if (error.name === 'AbortError') {
    throw new Error('Request timeout: 응답 시간이 5분을 초과했습니다.');
```

```

  }
  throw error;
} finally {
  clearTimeout(timeoutId);
}
}
```

C. 지능형 모드 전환

컨텍스트 기반 자동 판단

```

// 간단한 인사말 감지
function isSimpleGreeting(query: string): boolean {
  const greetings = ['안녕', '안녕하세요', 'hello', 'hi', '반가워'];
  const normalized = query.trim().toLowerCase().replace(/[\!?.,]/g, '');
  return greetings.some(greeting =>
    normalized.includes(greeting) && normalized.length <= greeting.length + 5
  );
}

// 모드별 최적화된 파라미터
const getOptimizedParams = (mode: string, hasKnowledgeBase: boolean) => {
  if (!hasKnowledgeBase) {
    return { temperature: 0.3, top_k: 0 }; // 일반 대화 모드
  }

  switch (mode) {
    case 'precise':
      return {
        similarity_threshold: 0.7, // 높은 정확도
        vector_similarity_weight: 0.8, // 벡터 검색 우선
        top_k: 50 // 정밀 검증
      };
    case 'quick':
      return {
        similarity_threshold: 0.3, // 빠른 응답

```

```

        top_k: 20 // 효율성 우선
    };
    case 'summary':
        return {
            similarity_threshold: 0.1, // 광범위 수집
            top_k: 100 // 포괄적 요약
        };
    }
};

```

🔗 4. 사용자 경험 최적화

A. 반응형 디자인

모바일/데스크톱 최적화

```

// 디바이스별 적응형 레이아웃
const shouldSwitchToChat = typeof window !== 'undefined' &&
    window.matchMedia('(max-width: 767px)').matches;

if (shouldSwitchToChat) {
    setCurrentView('chat'); // 모바일: 전체화면 채팅
} else {
    // 데스크톱: 사이드바 + 채팅 분할 화면
}

```

B. 접근성 고려사항

- 🗑 키보드 네비게이션: 모든 기능 키보드로 접근 가능
- 🔍 스크린 리더: 의미론적 HTML + ARIA 라벨
- 😴 고대비 모드: 다크/라이트 테마 지원
- 🖱 터치 친화적: 충분한 터치 영역 확보

C. 성능 최적화

```

// 컴포넌트 메모이제이션
const ModelBadge = memo(({ assistantId, currentMode }) => {
    return <Badge variant="secondary">Ollama@{currentMode}</Badge>;
});

// 무한 스크롤 + 가상화
const VirtualizedChatHistory = memo(({ messages }) => {
    return (
        <VirtualList
            itemCount={messages.length}
            renderItem={({ index }) => <ChatBubble message={messages[index]} />
        />
    );
});

```

```
);  
});
```

5. 실제 사용 시나리오

일반적인 사용 흐름:

-  파일 업로드: 드래그 앤 드롭으로 PDF 문서 업로드
-  자동 처리: 벡터화 완료 알림 (보통 30초-2분)
-  즉시 질문: "이 문서의 핵심 내용이 뭐야?"
-  정밀 응답: 출처 링크와 함께 상세 답변
-  품질 확인: 실시간 평가 점수 확인
-  추가 탐색: 출처 클릭으로 원본 문서 위치 확인
-  자동 저장: 모든 대화 내용 자동 보존

고급 사용 사례:

-  연구 보조: 여러 논문 동시 업로드 후 비교 분석
-  문서 요약: 긴 보고서를 핵심 포인트만 추출
-  사실 확인: 정밀 검증 모드로 정확성 검증
-  품질 관리: 평가 대시보드로 응답 품질 모니터링

이 시스템의 각 기능은 사용자 경험과 정보 정확성을 동시에 고려하여 설계되었으며, 특히 금융권에서 요구하는 높은 신뢰성과 추적 가능성을 제공합니다.

슬라이드 11: 아키텍처 설계 및 모델 구조도

1. 아키텍처 개요: 일반적인 챗봇과의 차별점

기존 챗봇 vs 우리 RAG 시스템

구분	일반적인 챗봇	우리 RAG 시스템
지식 처리	정적 지식베이스	동적 벡터DB + 실시간 문서 업로드
응답 품질	규칙 기반	AI 벡터 유사도 + Reranker
평가 시스템	수동 평가	4개 지표 자동 평가 (정확도/관련성/가독성/개인정보)
모드 전환	단일 모드	3개 전문 어시스턴트 (빠른응답/정밀검증/요약)

핵심 차별화 요소

```
// 멀티 어시스턴트 아키텍처  
const CHAT_MODES = [  
  { id: 'quick', name: '빠른 응답', icon: 'zap' },      // 일반 질의응답  
  { id: 'precise', name: '정밀 검증', icon: 'search' }, // 출처 확인 중심
```

```
{ id: 'summary', name: '요약', icon: 'file-text' } // 문서 요약 전문
];
```

🔗 2. 특정 모델 선정 이유: RAGFlow 선택 근거

RAGFlow vs 다른 RAG 프레임워크

☑ RAGFlow 선택 이유:

1. 엔터프라이즈급 API 완성도

- 5분 타임아웃 처리 + AbortController
- 1,031줄의 견고한 API 클라이언트 (ragflow.ts)
- 스트리밍 SSE 응답 지원

2. 고급 검색 기능

```
// 고급 검색 파라미터 지원
similarity_threshold: 0.2, // 유사도 임계값
vector_similarity_weight: 0.5, // 벡터/키워드 가중치
rerank_id: 'BAAI/bge-reranker-v2-m3', // 전문 리랭커
cross_languages: ['ko', 'en'] // 다국어 검색
```

3. 유연한 문서 처리

- 청크별 중요 키워드 자동 추출
- 데이터셋/문서/청크 3단계 관리
- 다양한 파서 지원 (PDF, Word, Excel 등)

우리 팀과의 적합성

- **금융권 특화:** 정밀한 출처 추적 및 검증 필요
- **대용량 처리:** 대량 문서의 효율적 벡터화
- **평가 중심:** 품질 측정 가능한 구조화된 응답

🔗 3. 핵심 기능별 세부 소개

A. 품질 평가 시스템 (LLM-as-a-Judge)

☑ 예, RAG 시스템이 실제로 작동해서 답변을 만들고 Judge가 평가합니다!

```
// 4개 지표 자동 평가 프레임워크
const EVALUATION_METRICS = {
  accuracy: {
    doc_match: 0.3, // 문서 일치도
    answer_from_retrieved: 0.5, // 검색된 내용 기반 답변
```



```

    no_hallucination: 0.2 // 환각 방지
  },
  relevance: "질문과의 연관성",
  readability: "가독성 및 간결성",
  privacy: "개인정보 노출 방지"
};

```

평가 프로세스:

1. 사용자 질문 → RAG 시스템 답변 생성
2. Ollama(Local LLM) → 4개 지표로 자동 평가
3. Gold Standard 데이터셋과 비교
4. 실시간 점수 표시 및 히스토리 저장

B. 벡터 DB 검색 최적화

```

// AI 점수 보존 + 상위권 하이라이트
function enhanceSearchResults(chunks: any[]): any[] {
  return chunks.map(chunk => ({
    ...chunk,
    similarity: chunk.similarity, // RAGFlow AI 점수 그대로 보존
    _enhancement_details: {
      original_score: chunk.similarity,
      ai_preserved: true // 인위적 조작 없이 AI 신뢰
    }
  })).sort((a, b) => b.similarity - a.similarity);
}

```

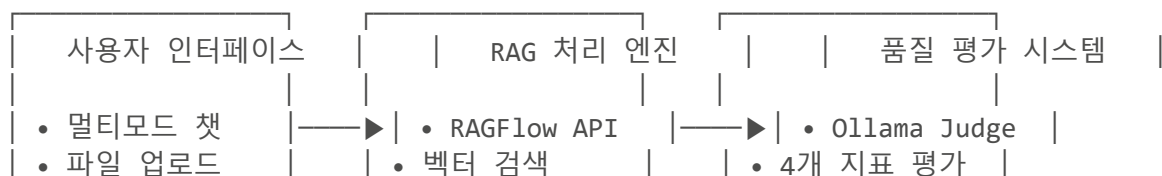
C. 동적 지식베이스 관리

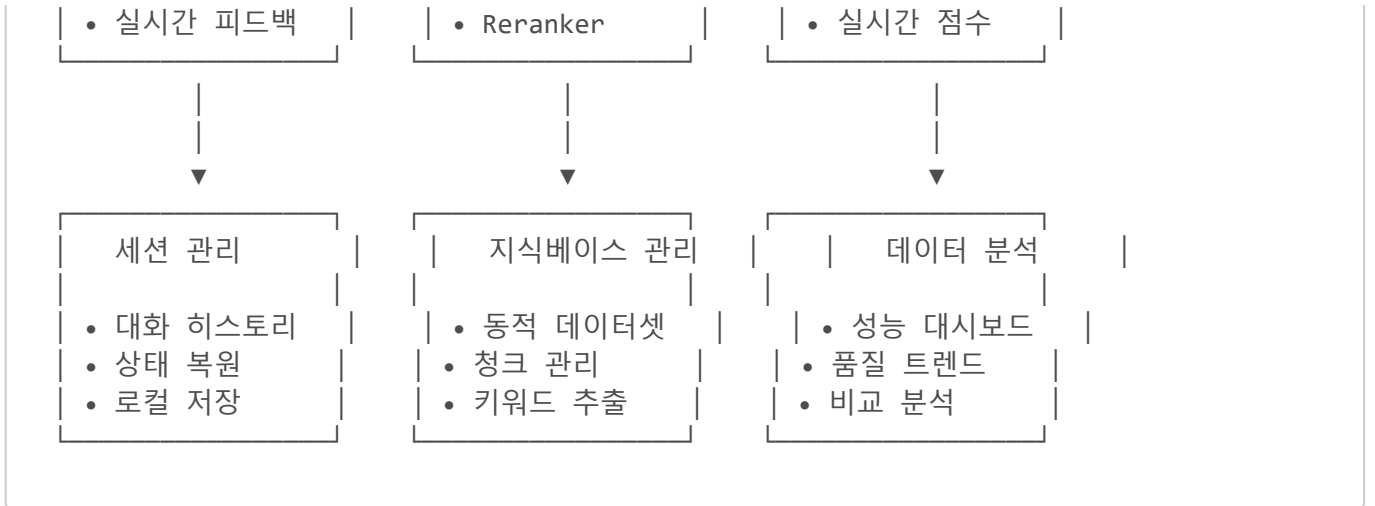
```

// 파일 업로드 → 임시 세션 생성
const handleFileUpload = async (files: File[]) => {
  const tempDataset = await createDataset(`temp_${sessionId}`);
  await uploadDocuments(tempDataset.id, files);
  await parseDocuments(tempDataset.id); // 자동 벡터화
  // 세션 종료 시 자동 삭제
};

```

📋 4. 전체 시스템 아키텍처





5. 기술적 우수성

엔터프라이즈급 안정성

- 타임아웃 관리: 5분 대용량 처리 지원
- 에러 처리: 다중 fallback 메커니즘
- 타입 안전성: 1,031줄 TypeScript API 클라이언트

고도화된 RAG 구현

- 동적 리랭킹: 환경변수로 모델 교체 가능
- 다국어 지원: cross_languages 검색
- 스코어 보존: AI 판단 신뢰, 인위적 조작 최소화

종합 품질 시스템

- 실시간 평가: 1,805줄 평가 엔진 (ollama.ts)
- 배치 처리: 대량 데이터 벤치마킹
- 로컬 LLM: 외부 API 의존성 없는 평가

6. 주요 특징 및 혁신점

A. 멀티 어시스턴트 시스템

- 빠른 응답: 일반적인 질의응답에 최적화
- 정밀 검증: 출처 확인 및 사실 검증 중심
- 요약 모드: 긴 문서의 핵심 내용 추출

B. 동적 지식베이스

- 실시간 파일 업로드: 즉석에서 새로운 지식 추가
- 임시 세션: 파일별 격리된 대화 환경
- 자동 벡터화: 업로드 즉시 검색 가능한 상태로 변환

C. 지능형 검색 최적화

- AI 점수 보존: RAGFlow + Reranker의 원본 점수 신뢰
- 상위권 하이라이트: 품질 높은 결과 자동 강조
- 메타데이터 필터링: 불필요한 관리정보 제거

D. 품질 보증 시스템

- 4개 핵심 지표: 정확도, 관련성, 가독성, 개인정보 보호
- 실시간 평가: 모든 응답에 대한 즉시 품질 측정
- 히스토리 추적: 시간에 따른 성능 변화 모니터링

7. 차별화 포인트 요약

특징	설명	기술적 구현
엔터프라이즈 안정성	5분 타임아웃, 에러 핸들링	AbortController, 다중 fallback
지능형 검색	AI 점수 보존, 동적 리랭킹	RAGFlow + BGE-reranker-v2-m3
품질 보증	4개 지표 실시간 평가	Ollama LLM-as-a-Judge
유연한 지식관리	동적 업로드, 임시 세션	격리된 데이터셋 생성/삭제
사용자 경험	멀티모드, 실시간 피드백	3개 전문 어시스턴트

이 시스템은 일반적인 챗봇을 넘어선 엔터프라이즈급 RAG 플랫폼으로, 특히 금융권의 정밀한 정보 검증과 품질 관리 요구사항에 최적화되어 있습니다.

8. 기술 스택 및 아키텍처 상세

Frontend Stack

- React + TypeScript: 타입 안전성과 컴포넌트 재사용성
- Vite: 빠른 개발 환경 및 빌드 최적화
- Tailwind CSS: 일관된 디자인 시스템
- Shadcn/ui: 접근성 높은 UI 컴포넌트

Backend Integration

- RAGFlow API: 엔터프라이즈급 RAG 엔진
- Ollama: 로컬 LLM 평가 시스템
- Vector Database: 실시간 의미론적 검색

Key Libraries & Tools

```
{
  "ragflow-client": "1,031줄 TypeScript 클라이언트",
  "ollama-integration": "1,805줄 평가 엔진",
  "session-management": "로컬스토리지 기반 상태 관리",
```

```
"streaming-sse": "실시간 응답 스트리밍"  
}
```

Development Features

- 환경변수 런타임 오버라이드: 배포 후 설정 변경 가능
- 프록시 서버 지원: CORS 및 네트워크 정책 우회
- 타입 안전 API: 컴파일 타임 에러 방지
- 에러 경계: 부분적 장애 시 graceful degradation

마지막 업데이트: 2024 년 12 월

작성자: RAG 시스템 개발팀